



Amazon ECR, ECS, Fargate, EKS

Lecture Slides

These lecture slides are provided for personal and non-commercial use only

Please do not redistribute or upload these lecture slides elsewhere.

Good luck on your exam!

Elastic Container Service (ECS)



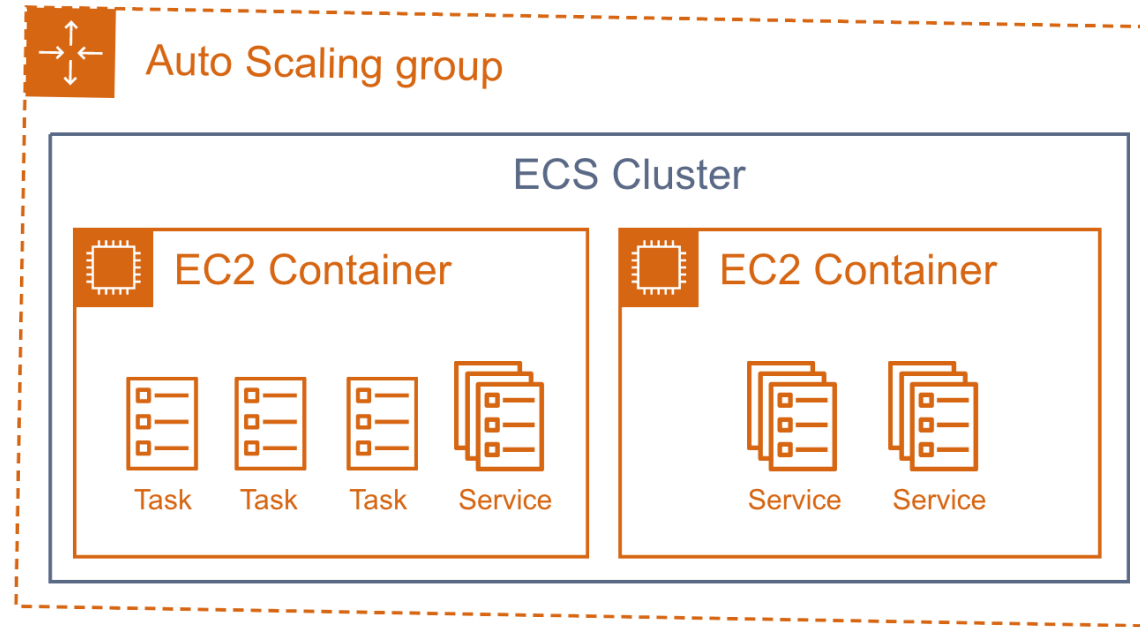
Fully-managed **container** orchestration service.
Highly secure, reliable, and scalable way to run containers

Think Docker as a Service

Introduction to ECS



Elastic Container Service (ECS) EC2 is a container orchestration service to run containers across a multiple EC2 machines managed in a cluster



Fargate is marketed as its own separate service but appears under ECS and most of the concepts that apply to ECS EC2 applies to Fargate

Components of ECS

Cluster

Multiple EC2 instances which will house the docker containers.

Task Definition

A JSON file that defines the configuration of (up to 10) containers you want to run

Task

Launches containers defined in Task Definition.
Tasks do not remain running once workload is complete.

Service

Ensures tasks remain running eg. Web app.

Container Agent

Binary on each EC2 instance which monitors, starts and stops tasks.

ECS Controller / Scheduler

Responsible for scheduling the deployment and placement of your containers, Replace unhealthy containers

- *You can create your own schedulers or use third-party schedulers*

ECS Execution Role

Execution Role is the role used to prepare or manage the container

Common permissions:

- Access to Secrets Manager or SSM Parameter Store
- Access to download private image from ECR
- Full Access to CloudWatch Logs

```
# CFN Example
ExecutionRole:
  Type: AWS::IAM::Role
  Properties:
    RoleName: CruddurServiceExecutionRole
    AssumeRolePolicyDocument:
      Version: '2012-10-17'
      Statement:
        - Effect: 'Allow'
          Principal:
            Service: 'ecs-tasks.amazonaws.com'
          Action: 'sts:AssumeRole'
    Policies:
      - PolicyName: 'cruddur-execution-policy'
        PolicyDocument:
          Version: '2012-10-17'
          Statement:
            - Sid: 'VisualEditor0'
              Effect: 'Allow'
              Action:
                - 'ecr:GetAuthorizationToken'
                - 'ecr:BatchCheckLayerAvailability'
                - 'ecr:GetDownloadUrlForLayer'
                - 'ecr:BatchGetImage'
                - 'logs:CreateLogStream'
                - 'logs:PutLogEvents'
              Resource: '*'
            - Sid: 'VisualEditor1'
              Effect: 'Allow'
              Action:
                - 'ssm:GetParameters'
                - 'ssm:GetParameter'
              Resource: '!Sub
'arn:aws:ssm:${AWS::Region}:${AWS::AccountId}:parameter/cru
ddur/${ServiceName}/*'
        ManagedPolicyArns:
          - arn:aws:iam:aws:policy/CloudWatchLogsFullAccess
```

ECS Task Role

Task Role is the role that is used by the running compute of the container

Common permissions:

- Access to SSM Messages for ECS Exec
- CloudWatch Logs Full access so container can log
- XRay Daemon Write Access so Xray can be used for traceability

```
TaskRole:
  Type: AWS::IAM::Role
  Properties:
    RoleName: CruddurServiceTaskRole
    AssumeRolePolicyDocument:
      Version: '2012-10-17'
      Statement:
        - Effect: 'Allow'
          Principal:
            Service: 'ecs-tasks.amazonaws.com'
          Action: 'sts:AssumeRole'
    Policies:
      - PolicyName: 'cruddur-task-policy'
        PolicyDocument:
          Version: '2012-10-17'
          Statement:
            - Sid: 'VisualEditor0'
              Effect: 'Allow'
              Action:
                - ssmessages:CreateControlChannel
                - ssmessages:CreateDataChannel
                - ssmessages:OpenControlChannel
                - ssmessages:OpenDataChannel
              Resource: '*'
        ManagedPolicyArns:
          - arn:aws:iam::aws:policy/CloudWatchLogsFullAccess
          - arn:aws:iam::aws:policy/AWSXRayDaemonWriteAccess
```

ECS Capacity Providers

Amazon ECS capacity providers manage the **scaling of infrastructure for tasks** in your clusters.

Each cluster can have one or more capacity providers and an optional capacity provider strategy.

Fargate has two predefined capacity providers (FARGATE and FARGATE SPOT) or you can create your own custom capacity provider

```
aws ecs create-capacity-provider \  
--name FargateCapacityProvider \  
--auto-scaling-group-provider managedScaling={"status": "ENABLED", "targetCapacity": 100} \  
--auto-scaling-group-provider managedTerminationProtection="ENABLED"  
  
# At the cluster level  
aws ecs put-cluster-capacity-providers \  
--cluster example-cluster \  
--capacity-providers FargateCapacityProvider FARGATE FARGATE_SPOT \  
--default-capacity-provider-strategy capacityProvider="FARGATE_SPOT",weight=1,base=0  
capacityProvider="FARGATE",weight=1  
  
# At the task level  
aws ecs create-service \  
# ...  
--capacity-provider-strategy capacityProvider="FARGATE_SPOT",weight=1  
capacityProvider="FARGATE",weight=1
```

ECS Capacity Providers

For ECS EC2 you create an Auto Scaling Group and associate that with your custom capacity provider. You then attach then custom capacity provider to ESC EC2

```
aws autoscaling create-auto-scaling-group --min-size 1 --max-size 10 --desired-capacity 2 \
# and other ASG flags...

aws ecs create-capacity-provider \
--name MyEC2CapacityProvider \
--auto-scaling-group-provider autoScalingGroupArn=ASG_ARN,managedScaling=
{"status":"ENABLED","targetCapacity":75},managedTerminationProtection="ENABLED"

# At the cluster level
aws ecs put-cluster-capacity-providers \
--cluster my-cluster \
--capacity-providers MyEC2CapacityProvider \
--default-capacity-provider-strategy capacityProvider="MyEC2CapacityProvider",weight=1,base=1

# At the task level
aws ecs create-service \
# ...
--capacity-provider-strategy capacityProvider="MyEC2CapacityProvider",weight=1,base=1
```


ECS Task Lifecycle

ACTIVATING

perform additional steps after the task is launched but before the task is running

PENDING

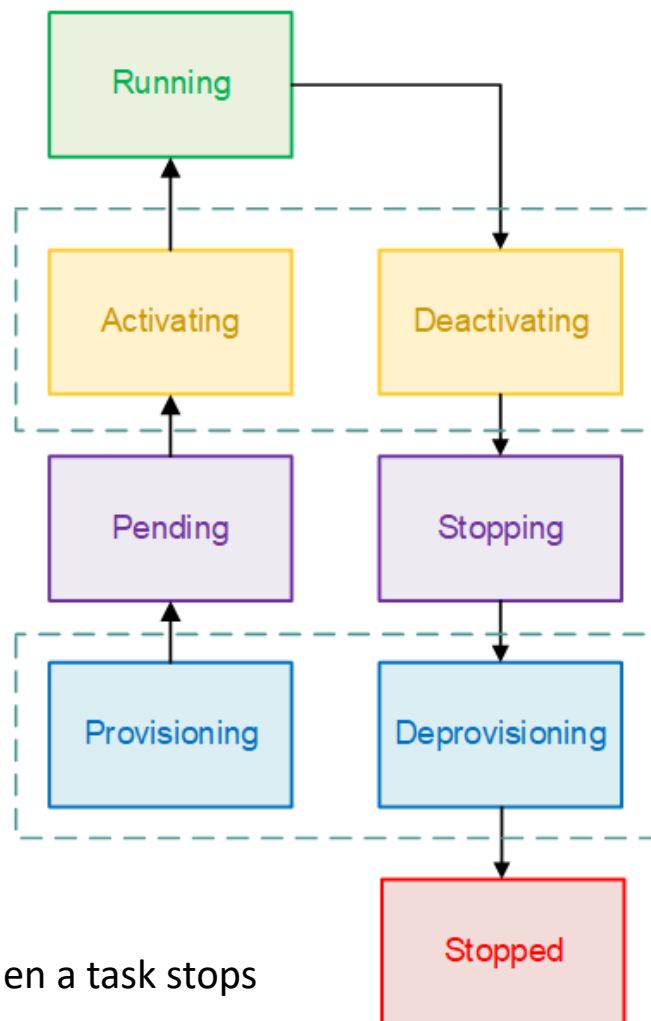
waiting on the container agent to take further action

PROVISIONING

additional steps before the task is launched
Eg. Launching an Attaching ENIs

DELETED

transition state when a task stops



RUNNING

task is successfully running

DEACTIVATING

perform additional steps before the task is stopped

STOPPING

waiting on the container agent to take further action

DEPROVISIONING

additional steps after the task has stopped
Eg. Detaching and deleteing ENIs

STOPPED

task has been successfully stopped.

Task Definition JSON File Example

- **Family** — a way to group similar task definitions (its how versioning works)
- **Execution Role** — the role used to prepare or manage the container
- **Task Role** — the role that is used by the running compute of the container
- **Network Mode**
 - Host — most basic mode, connect directly to the host machine
 - Bridge — isolate between containers but they can still commute with each other
 - AWSVPC — creates an ENI in your VPC with a private IP address
 - **Fargate can only use AWSVPC mode**
 - None — disable networking
- **CPU and Memory** — how much memory and compute
- **Requires Compatibilities** — EC2, FARGATE, EXTERNAL
- **Container Definition** — Defines the connection of containers to be provisioned on the compute

```
{
  "family": "backend-flask",
  "executionRoleArn": "arn:aws:iam::38754",
  "taskRoleArn": "arn:aws:iam::3875430594",
  "networkMode": "awsvpc",
  "cpu": "256",
  "memory": "512",
  "requiresCompatibilities": [
    "FARGATE"
  ],
  "containerDefinitions": [...]
}
```

Incomplete example

Task Definition JSON File Example

- **Name:** name of the container
- **Image:** a URI to the container image eg. ECR, DockerHub
- **Essential:** There always has to be one essential container, if this container fails all containers fail
- **Health Check:** Perform a healthcheck
- **Port Mappings:** Map the guest to host ports
- **Log Configuration:** Write logs to AWS CloudWatch
- **Environment:** Env Vars you want to set for your container
- **Secrets:** secrets pulled from Secrets Manager or SSM Parameter Store

```
{ // ...
  "containerDefinitions": [
    {
      "name": "backend-flask",
      "image": "387543059434.dkr.ecr.ca-central-1.amazonaws.com/backend-flask",
      "essential": true,
      "healthCheck": {
        "command": [
          "CMD-SHELL",
          "python /backend-flask/bin/health-check"
        ],
        "interval": 30, "timeout": 5, "retries": 3, "startPeriod": 60
      },
      "portMappings": [
        {
          "name": "backend-flask",
          "containerPort": 4567,
          "protocol": "tcp",
          "appProtocol": "http"
        }
      ],
      "logConfiguration": {
        "logDriver": "awslogs",
        "options": {
          "awslogs-group": "cruddur",
          "awslogs-region": "ca-central-1",
          "awslogs-stream-prefix": "backend-flask"
        }
      },
      "environment": [
        {"name": "AWS_DEFAULT_REGION", "value": "ca-central-1"}
      ],
      "secrets": [
        {"name": "AWS_ACCESS_KEY_ID",
         "valueFrom": "arn:aws:ssm:ca-central-1:387543059434:parameter/..."}
      ]
    }
  ]
}
```

Port Mappings

```
"portMappings": [  
  {  
    "containerPort": 3000,  
    "hostPort": 80,  
    "protocol": "tcp"  
  }  
]
```



Bridge Mode

- container (guest) port maps to hosts different port
- The guest and host ports can be different

```
"portMappings": [  
  {  
    "containerPort": 80,  
    "protocol": "tcp"  
  }  
]
```



Host Mode

- container (guest) port directly maps to the same host's port.
- The guest and host ports will be the same
 - You don't need to defined the host port because it will be the same value

```
"portMappings": [  
  {  
    "containerPort": 80,  
    "protocol": "tcp"  
  }  
]
```



AWSVPC Mode

- Gives the container its own network interface with a direct public IP
- The guest and host port will be the same
 - You don't need to defined the host port because it will be the same value

ECS Exec

ECS Exec allows you to **directly interact with containers** without needing to first interact with the host container operating system, open inbound ports, or manage SSH keys.

ECS Exec works with both ECS EC2 containers and ECS Fargate containers

- ECS Exec commands are run as root
- ECS Exec commands cannot be executed via the AWS Management Console
 - You must use a terminal
- ECS Exec session has an idle timeout time of 20 minutes
- ECS Exec must be turned on at the launch of a task
 - ECS Exec cannot be turned on for existing run tasks

Prerequisites:

- AWS CLI Installed
- Sessions Manager Plugin Installed
- The Task role must have **permission**
- You must meet version requirements for ECS or Fargate

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": [
      "ssmmessages:CreateControlChannel",
      "ssmmessages:CreateDataChannel",
      "ssmmessages:OpenControlChannel",
      "ssmmessages:OpenDataChannel"
    ],
    "Resource": "*"
  }]
}
```



ECS Exec

In your Task Definition to avoid Zombie SSM agent children set `initProcessEnabled` for Linux

```
"containerDefinitions": [{  
  // ....  
  "linuxParameters": {  
    "initProcessEnabled": true  
  }  
}],
```

Turn on ECS Exec when creating the service

```
aws ecs create-service \  
# .... other flags \  
--enable-execute-command
```

Use ECS Exec to remotely execute a command

```
aws ecs execute-command \  
--cluster cluster-name \  
--task task-id \  
--container container-name \  
--interactive \  
--command "/bin/sh"
```

Log Configuration

When configuring your containers in your Task Definition you can set a log driver.

A log driver tells where the container should log.

awslogs will log to CloudWatch Logs

```
"logConfiguration": {  
  "logDriver": "awslogs",  
  "options": {  
    "awslogs-group": "my-ecs-logs",  
    "awslogs-region": "us-west-2",  
    "awslogs-stream-prefix": "ecs"  
  }  
}
```

There are other third-party log drivers you can use.

ECS EC2 log driver support

- awslogs
- fluentd,
- gelf,
- json-file,
- journald,
- logentries
- syslog,
- splunk,
- awsfirelens

Fargate log driver support:

- awslogs
- splunk
- awsfirelens

Awslogs by default is blocking and you may want to configure it for nonblock or instead use AWS Firelens

AWS FireLens works with either Fluent Bit or Fluentd. That run as a sidecar container in the same ECS task. Firelens is used to avoid backpressure

ECS Service Connect

ECS Service Connect makes it easy to setup a service mesh for service-to-service communication

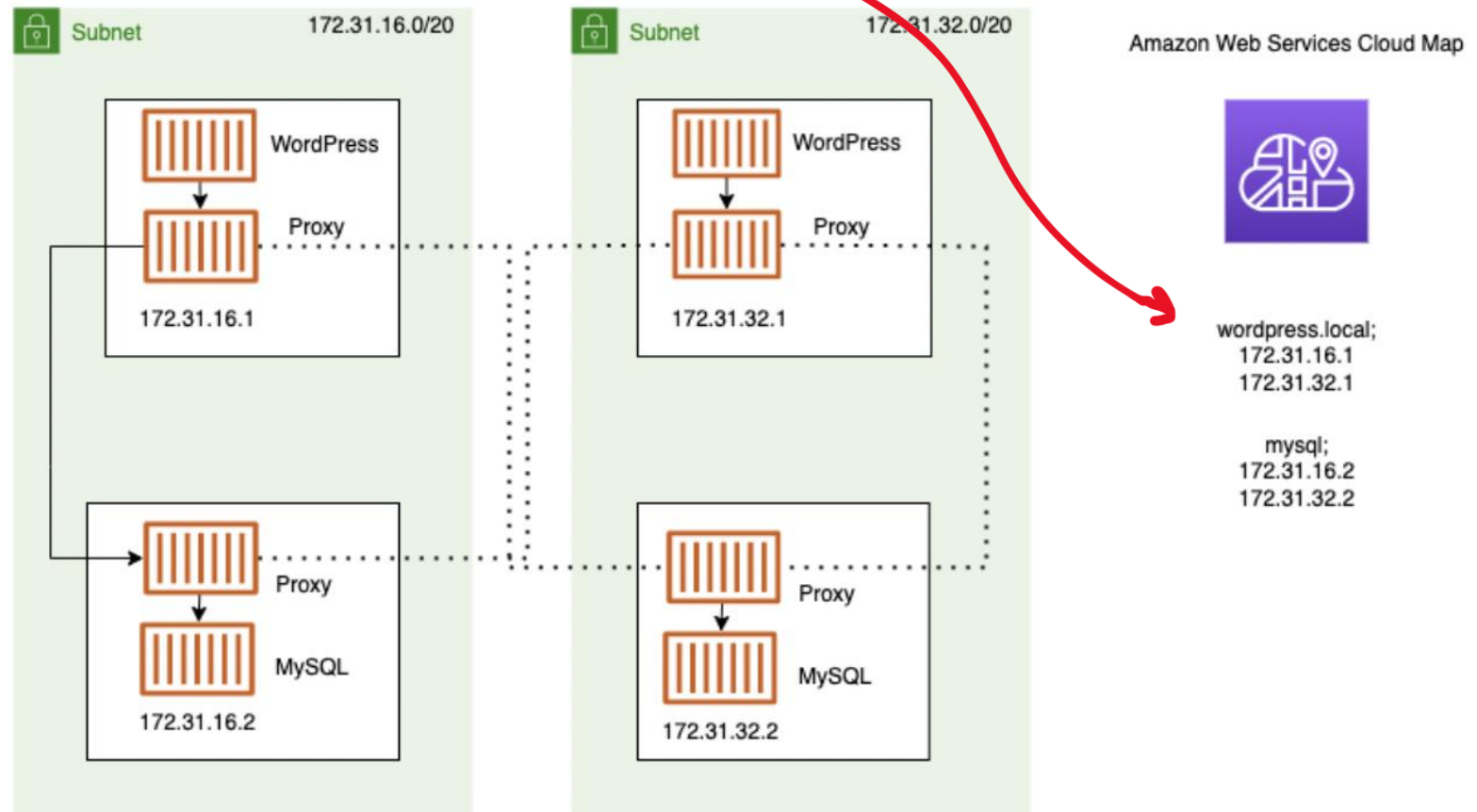
Service Connect is an evolution of App Mesh, abstracting a lot of the configuration between App Mesh, Cloud Map and ELB

Service Connect will deploy a sidecar proxy container eg. Envoy

You can use the **service discovery name** to easily talk to other services

Service Connect Features:

- Service Discovery
- consistent approach to handle service-to-service communications
- Encrypts data in transit between services using TLS
- Telemetry data in the ECS Console and CloudWatch
- Traffic health checks
- Automatic retries
- Rolling deployments



ECS Service Connect

When you create your cluster you'll defined a ServiceConnectDefaults.

*This creates a **CloudMap Namespace***

```
Resources:
  FargateCluster:
    Type: AWS::ECS::Cluster
    Properties:
      ClusterName: !Sub "${AWS::StackName}FargateCluster"
      CapacityProviders:
      ServiceConnectDefaults:
        Namespace: cruddur
```

When you create your service you configure it for Service Connect.

Providing the Namespace.

Its Discover Name and Port Name

```
Resources:
  FargateService:
    Type: AWS::ECS::Service
    Properties:
      # other proprerities....
      ServiceConnectConfiguration:
        Enabled: true
        Namespace: "cruddur"
        # TODO - If you want to log
        # LogConfiguration
      Services:
        - DiscoveryName: backend-flask
          PortName: backend-flask
          ClientAliases:
            - Port: !Ref ContainerPort
```

ECS Optimized AMI

Amazon ECS-optimized AMIs are preconfigured with the requirements and recommendations to run your container workloads.

When you launch EC2 instance via ECS EC2 Management Console it will automatically use an Amazon EC2 Optimize AMI **by default**

Features of ECS Optimized AMIs:

- Comes with Docker installed
- Comes with ECS Container Agent Installed
- OS-level optimized for containers
- There is a variant of ECS Optimized with GPUs

To change the AMI (if you needed the GPU variant) would be adjusted in the launch template

```
aws ec2 create-launch-template \  
  --launch-template-name "ECSEc2LaunchTemplate" \  
  --version-description "Version1" \  
  --launch-template-data \  
    '{  
      "ImageId": "ami-0abcdef1234567890",
```

ECS Optimized Bottlerocket AMI



Bottlerocket is a Linux-based open-source operation system that is purpose-built by AWS for running containers on Virtual Machines or bare metal hosts

- Bottlerocket doesn't include a package manager
- Software can only be run as containers
- Updates are both applied and can be rolled back in a single step, which reduces the likelihood of update errors.

Bottlerocket AMIs also don't support the following services and features:

- ECS Anywhere
- Service Connect
- Amazon EFS in encrypted mode and awsvpc network mode

Registering ECS EC2 Instances to Cluster

To add an ECS EC2 instance to a cluster you have to set this

```
# userdata.toml
[settings.ecs]
cluster = "my-cluster-name"
```



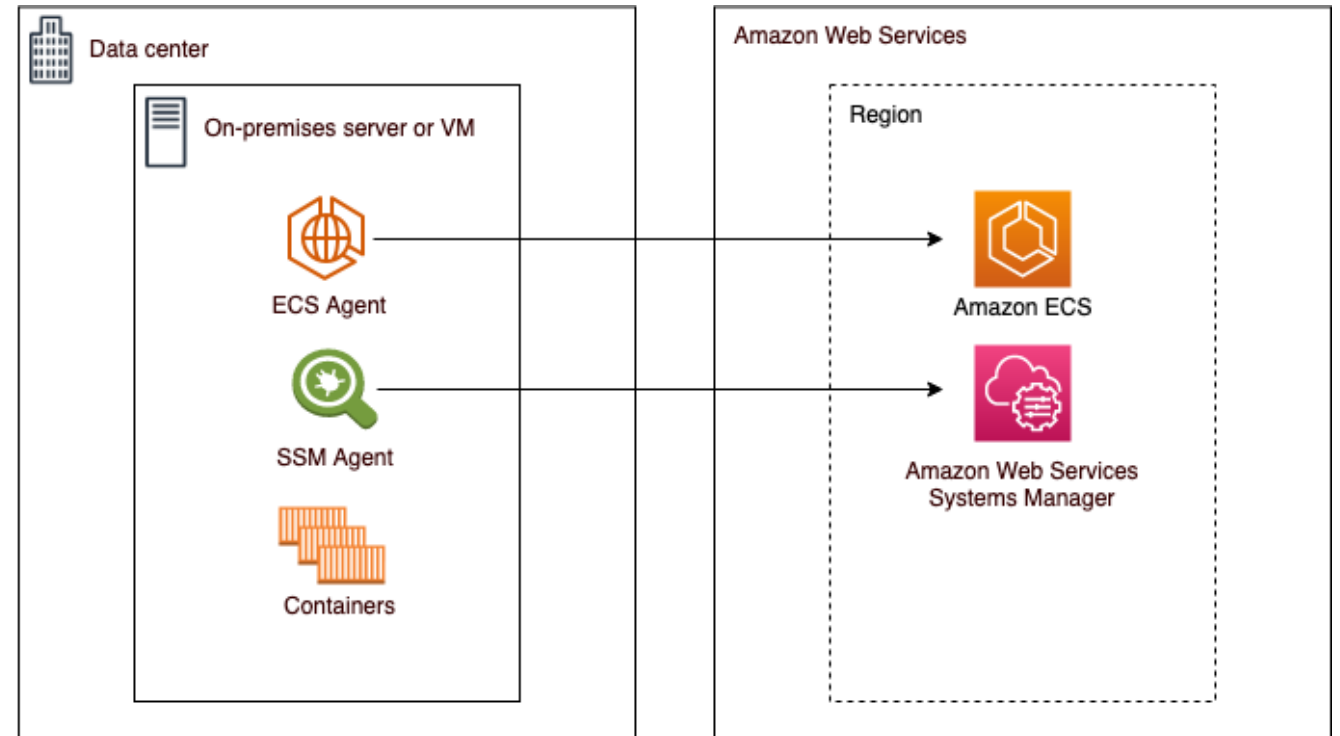
```
aws ec2 run-instances \
--key-name ecs-bottlerocket-example \
--subnet-id subnet-08fc749671b2d077c \
--image-id <bottlerocket_ami> \
--instance-type t3.large \
--region ca-central-1 \
--user-data file://userdata.toml \
--iam-instance-profile Name=ecsInstanceRole
```

ECS Anywhere



Amazon ECS Anywhere allows you to register external VMs residing from your on-premise network to your ECS Cluster.

- \$0.01025 per hour for each managed ECS Anywhere on-premises instance
- You can register an external instance to a single cluster
- external instances require an IAM role that allows them to communicate with AWS APIs
- ECS Exec is supported on external instances
- awsvpc network mode isn't supported.
- Service load balancing isn't supported.
- Service discovery isn't supported.
- ECS capacity providers aren't supported.
- ECS Anywhere uses the launch type EXTERNAL
- SELinux isn't supported
- EFS volumes aren't supported
- You can run ECS Anywhere on Windows but you need a Windows License



ECS Anywhere

To install you need to

- create SSM Activation pair,
- download the install script to your machine
- run the install script.

```
# Create an Systems Manager activation pair
aws ssm create-activation --iam-role ecsAnywhereRole | tee ssm-activation.json

# Download Install Script
curl --proto "https" -o "/tmp/ecs-anywhere-install.sh" \
  "https://amazon-ecs-agent.s3.amazonaws.com/ecs-anywhere-install-latest.sh"

# Run the Install Script
sudo bash /tmp/ecs-anywhere-install.sh \
  --region $REGION \
  --cluster $CLUSTER_NAME \
  --activation-id $ACTIVATION_ID \
  --activation-code $ACTIVATION_CODE
```

The Install scripts runs and starts the ECS Service agent which you can manage via **systemctl**

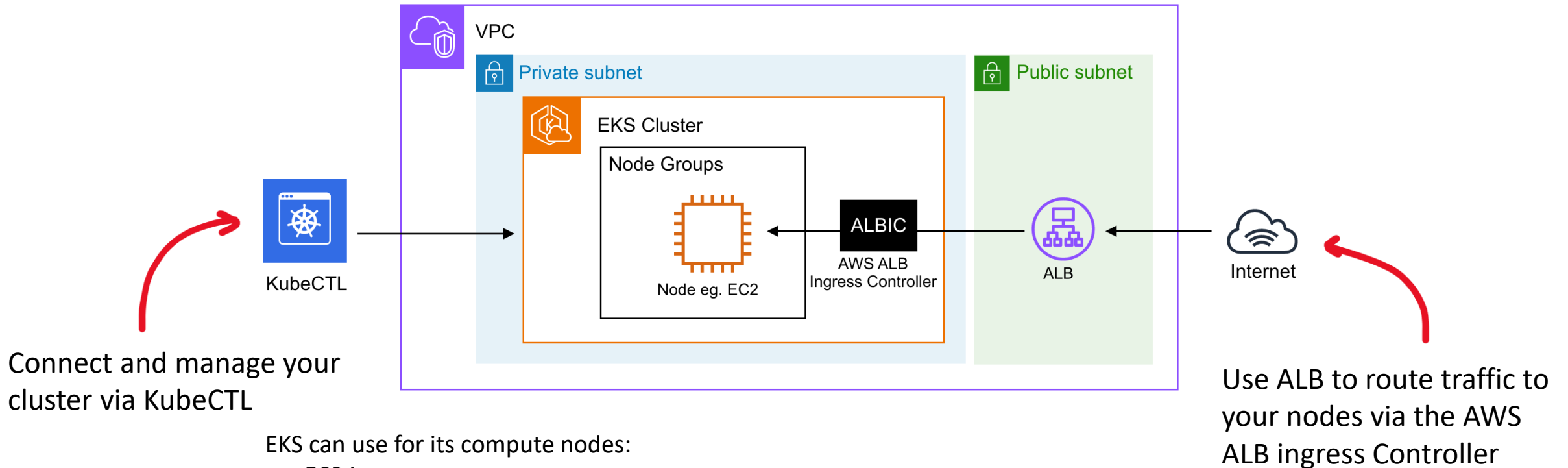
```
sudo systemctl start ecs.service
```



EKS Cloud



Amazon Elastic Kubernetes Service (Amazon EKS) is a managed service that eliminates the need to install, operate, and maintain your own Kubernetes control plane on AWS



EKS can use for its compute nodes:

- EC2 instances
 - Managed Node groups — Auto scaling is fully managed by AWS
 - Self-Managed Node Groups — Customer can self-manage scaling using EC2 Auto Scaling Groups
 - Karpenter — cloud-native open-source autoscaler
- Fargate instances
- External instances (on-premise)

EKS Add-Ons

AWS Managed Add Ons

- **Amazon VPC CNI plugin for Kubernetes**
 - Enable pod networking within your cluster.
- **CoreDNS**
 - Enable service discovery within your cluster.
- **Kube-proxy**
 - Enable service networking within your cluster.
- **Amazon EKS Pod Identity Agent**
 - Install EKS Pod Identity Agent to use EKS Pod Identity to grant AWS IAM permissions to pods through Kubernetes service accounts
- Amazon GuardDuty agent
- Amazon EBS CSI Driver
- Amazon EFS CSI Driver
- Mountpoint for Amazon S3 CSI Driver
- CSI snapshot controller
- AWS Distro for OpenTelemetry
- Amazon CloudWatch Observability agent

Third-Party Add Ons

- | | |
|----------------|--------------------------------|
| • Accuknox | • Kasten |
| • NetApp | • Kong |
| • Calyptia | • LeakSignal |
| • Cribl | • New Relic |
| • Dynatrace | • Rafay |
| • Datree | • Solo.io |
| • Datadog | • Stormforge |
| • Groundcover | • Splunk |
| • Grafana Labs | • Teleport |
| • HA Proxy | • Tetrade |
| • Kpow | • Upbound Universal Crossplane |
| • Kubecost | • Upwind |

EKS Connector

Amazon EKS Connector to register and connect any conformant Kubernetes cluster to AWS and visualize it in the Amazon EKS console. *(Bring your own K8 cluster to EKS)*

You install EKS Connector via helm in your targeted cluster

```
helm -n eks-connector install eks-connector \
oci://public.ecr.aws/eks-connector/eks-connector-chart \
--set eks.activationCode="<your-activation-code>" \
--set eks.activationId="<your-activation-id>" \
--set eks.agentRegion="<your-region>"
```

EKS CTL

EKS CTL is a CLI Tool for easily setting up K8s clusters on AWS.

EKS can deploy:

- EC2-backed nodes
- Fargate-backed nodes
- To a private cluster on AWS Outposts

```
apiVersion: eksctl.io/v1alpha5
kind: ClusterConfig
```

```
metadata:
  name: basic-cluster
  region: eu-north-1
```

```
nodeGroups:
- name: ng-1
  instanceType: m5.large
  desiredCapacity: 10
- name: ng-2
  instanceType: m5.xlarge
  desiredCapacity: 2
```

```
eksctl create cluster
```

By default, EKS CTL will use the following settings:

- auto-generated name, eg., fabulous-mushroom-1527688624
- two m5.large worker nodes
- EC2 Instances configured with AWS EKS AMI
- Deployed in us-west-2 region
- a dedicated VPC

EKS CTL can be configure via a **configuration file**:

```
eksctl create cluster -f cluster.yaml
```

EKS Distro



EKS Distro (EKS-D) is a **Kubernetes distribution based on and used by EKS** to create reliable and secure K8s clusters.

<https://distro.eks.amazonaws.com>

Use-cases:

Hybrid Deployments — consistency between AWS and on-prem.
Development and Testing — identical prod and dev environment.
AWS Services Extension — AWS integration to on-premises setups.

Installing Methods:

- Community: kubeadmn, kinit, kops
- Third-party partners eg. Pulumi, DataDog, SysDig, Kubestack
- EKS Anywhere

EKS-D comes with the following **components**:

- CNI plugins
- CoreDNS
- etcd
- CSI Sidecars
- aws-iam-authenticator
- Kubernetes Metrics Server
- Kubernetes



A supported installation method for EKS-D is available with **EKS Anywhere (EKS-A)**.

EKS Anywhere



Amazon EKS Anywhere (EKS-A) is a deployment option for Amazon EKS to easily create and operate K8s clusters on-premises with your own VMs or bare metal hosts.



EKS-A deploys the Amazon EKS Distro as the K8s distribution.



EKS Anywhere allow you to manage your deployed clusters from the **AWS Management Console**

Admin Machine is required to run cluster lifecycle operation

- Does not have to continuously run
- critical cluster artifacts are saved to the admin machine on creation eg kubeconfig file, SSH keys, the full cluster specification yaml

EKS-Anywhere is open-source is free.

EKS Anywhere Enterprise Subscriptions for 24/7

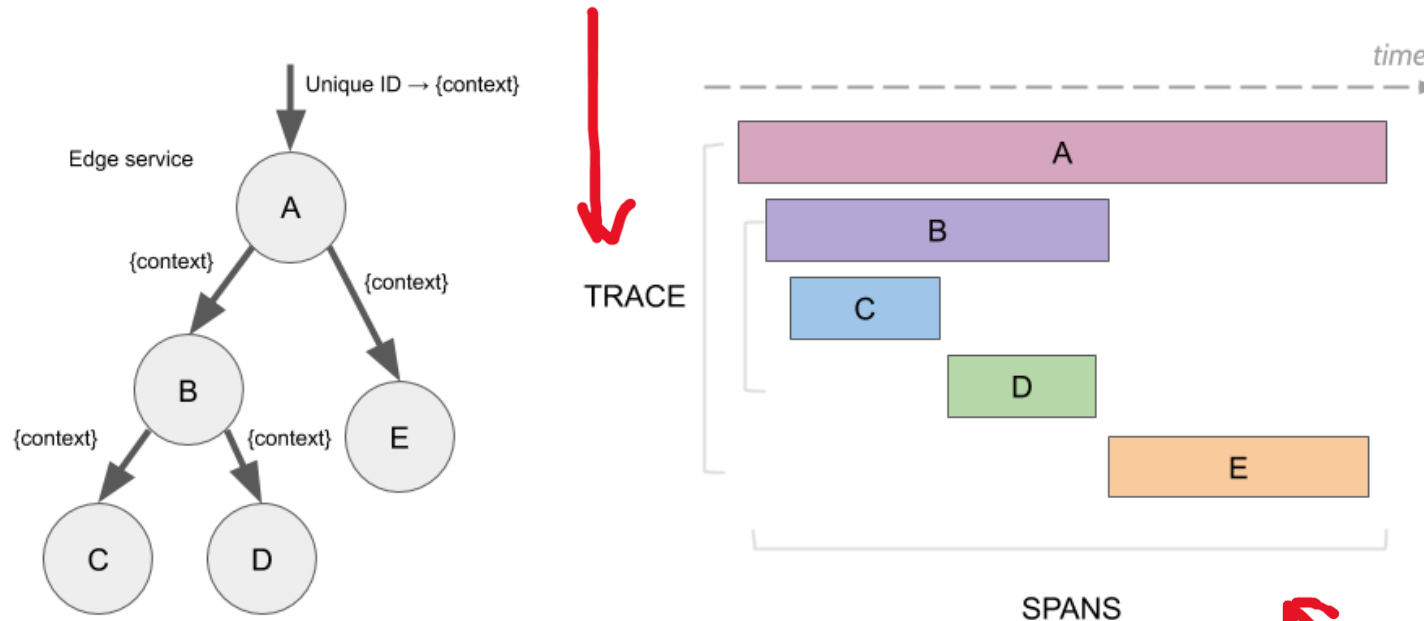
- 1-year term at \$24,000 per cluster
- 3-year term at \$18,000 per cluster per year

A cluster can be deployed to

- VMWare vSphere
- Bare Metal (using Tinkerbell)
- AWS Snowball Edge
- Apache CloudStack
- Nutanix
- **Docker** (development clusters)

Traces and Spans

A **trace** is a data/execution path through the system, and can be thought of as a **directed acyclic graph (DAG)** of spans



A **span** represents a logical unit of work in Jaeger that has an operation name, the start time of the operation, and the duration.

Spans may be nested and ordered to model causal relationships.

Open Telemetry



Open Telemetry (OTEL) is a collection of open-source tools, APIs and SDKs to instrument, generate, collect, and export telemetry data

Open Telemetry **standardizes** the way telemetry data (metrics, logs, and traces) are generated and collected.

Wire protocol

a wire protocol refers to a way of getting data from point to point. Eg. SOAP, AMQP



Open Telemetry – Instrumentation

Instrumentation is the act of embedding a monitoring library into your existing application in order to capture monitoring data such as: metrics, traces, or logging

Open Telemetry supports a variety of languages:

- C++
- .NET
- Erlang / Elixir
- Go
- Java
- Javascript
- Php
- Python
- Ruby
- Rust
- Swift

For certain frameworks there
plug-and-play libraries to
quickly instrument your apps:

- Spring
- ASP.NET Core
- Express
- Quarkus

```
# Require otel-ruby
require 'opentelemetry/sdk'

# Export traces to console by default
ENV['OTEL_TRACES_EXPORTER'] ||= 'console'

# configure SDK with defaults
OpenTelemetry::SDK.configure

@tracer = OpenTelemetry.tracer_provider.tracer('sinatra', '1.0')

OpenTelemetry::Context.with_current(context) do
  # Span kind MUST be `:server` for a HTTP server span
  @tracer.in_span(
    span_name,
    attributes: {
      'component' => 'http',
      'http.method' => env['REQUEST_METHOD'],
      'http.route' => env['PATH_INFO'],
      'http.url' => env['REQUEST_URI'],
    },
    kind: :server
  ) do |span|
    # Run application stack
    status, headers, response_body = @app.call(env)

    span.set_attribute('http.status_code', status)
  end
end
```

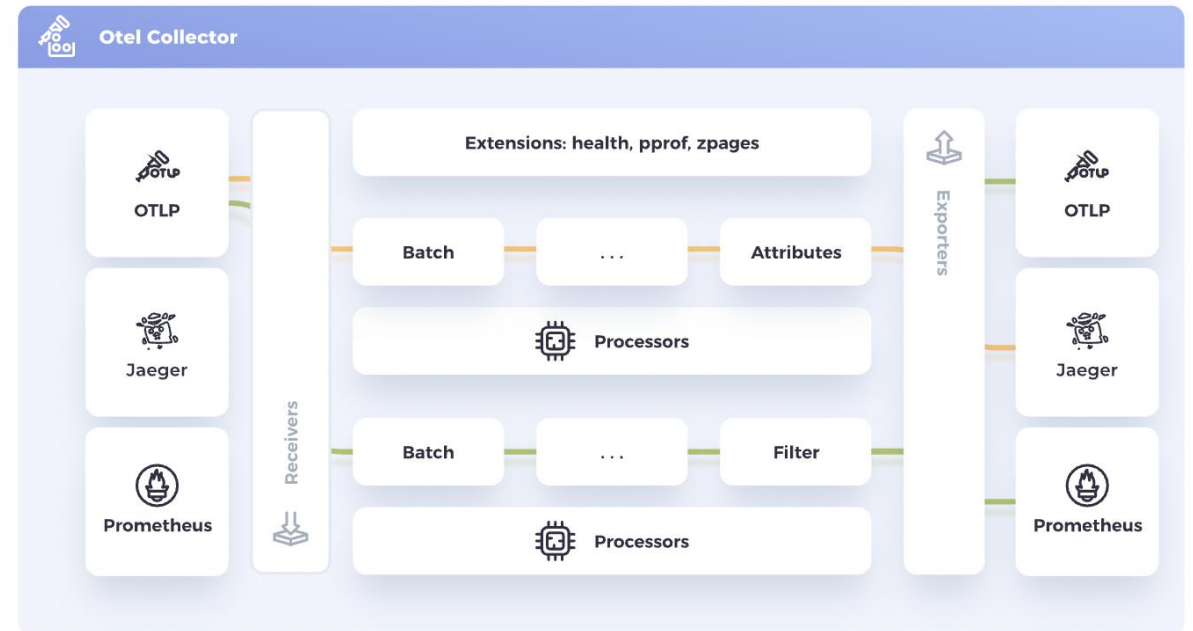
Open Telemetry - Collector

The Open Telemetry collector is an agent installed on the target machine or as a dedicated server and is a Vendor-agnostic way to receive, process, and export telemetry data

It removes the need to run, operate, and maintain multiple agents/collectors.

This works with improved scalability and supports open-source observability data formats (e.g. Jaeger, Prometheus, Fluent Bit, etc.) sending to one or more open-source or commercial back-ends.

The local Collector agent is the default location to which instrumentation libraries export their telemetry data.

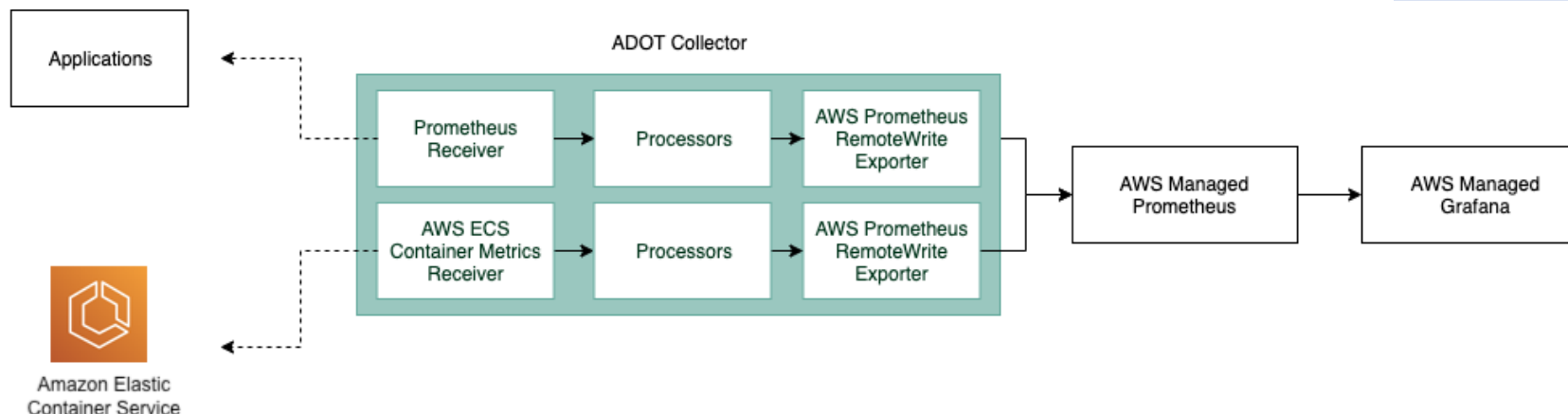


AWS Distro for OpenTelemetry



AWS Distro for OpenTelemetry (ADOT) is a secure, production-ready, AWS-supported distribution of the OpenTelemetry project.

<https://aws-otel.github.io/>



Send correlated logs, metrics, and traces to or from more observability backends such as:

- Amazon Managed Service for Prometheus (AMP)
- Amazon Managed Streaming for Apache Kafka (MSK)
- Amazon CloudWatch
- AWS X-Ray
- Amazon Open Search
- any OpenTelemetry Protocol (OTLP) compliant backend

Observe apps running in:

- EC2
- ECS EC2
- Fargate
- EKS
- AWS App Runner
- AWS Lambda
- On-premise

Prometheus



Prometheus is **an open-source systems monitoring and alerting toolkit** originally built at SoundCloud.

Prometheus collects and stores its metrics as time series data.

Prometheus is a **time-series database**



Prometheus's main features are:

- A multi-dimensional data model with time series data identified by metric name and key/value pairs
- PromQL, a flexible query language to leverage this dimensionality
- No reliance on distributed storage; single server nodes are autonomous
- Time series collection happens via a pull model over HTTP
- Pushing time series is supported via an intermediary gateway
- Targets are discovered via service discovery or static configuration
- Multiple modes of graphing and dashboarding support

Prometheus values reliability.

You can always view what statistics are available about your system, even under failure conditions.

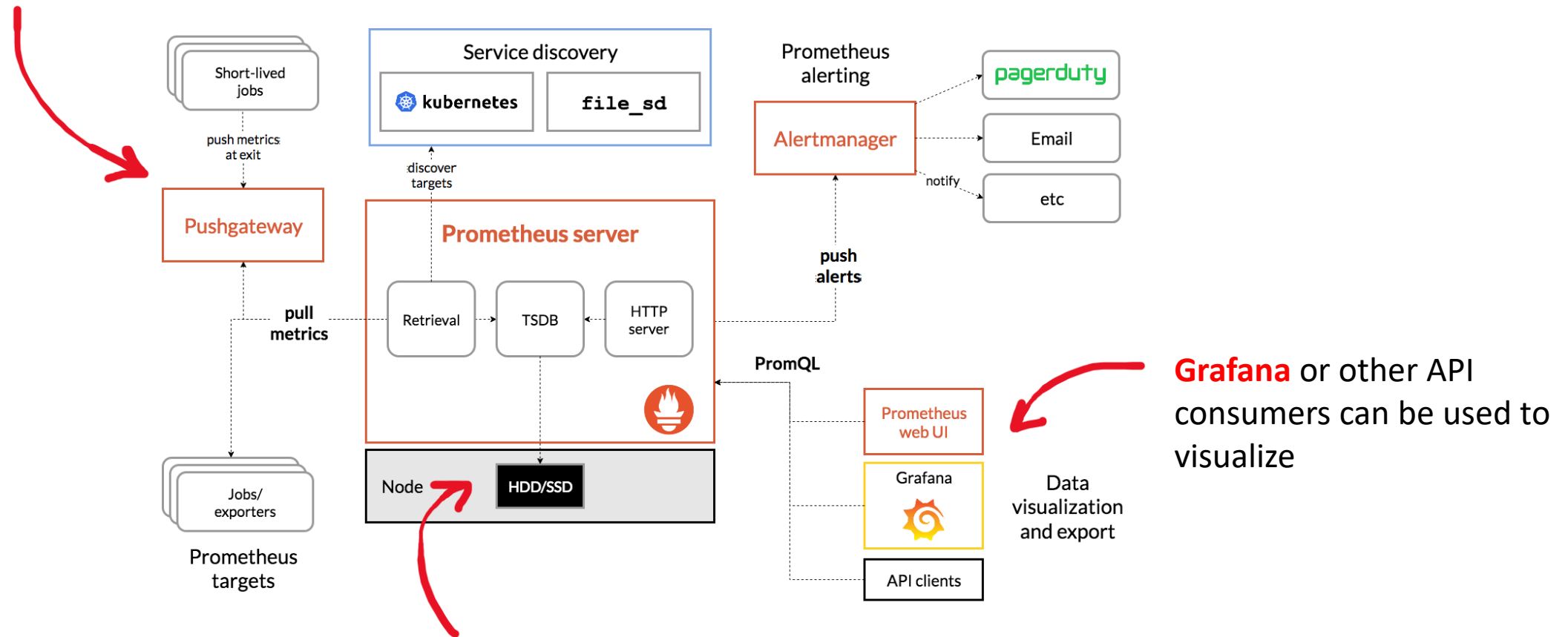


If you need 100% accuracy, such as for per-request billing, Prometheus is not a good choice, as the collected data will likely not be detailed and complete enough.

In such a case, you would be best off using some other system to collect and analyze the data for billing and Prometheus for the rest of your monitoring.

Prometheus

Prometheus scrapes metrics from **instrumented jobs**, either directly or via an intermediary push gateway for short-lived jobs.



It stores all scraped samples locally and runs rules over this data to either aggregate and record new time series from existing data or generate alerts

Amazon Managed Service for Prometheus

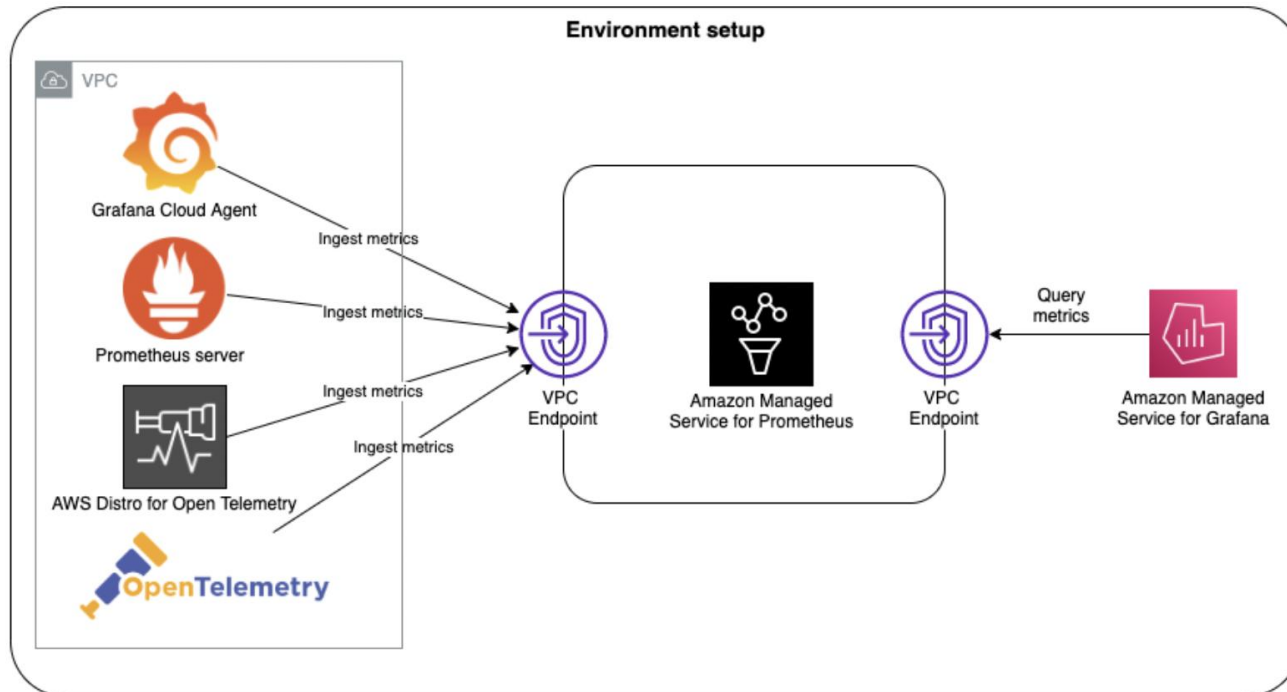


Amazon Managed Service for Prometheus (AMP) is a Prometheus-compatible monitoring service for container infrastructure and application metrics.

AMP is a fully-managed Prometheus Server Environment



AMP makes it easy for customers to securely monitor container environments at scale



You can **use Amazon Managed Service for Grafana (AMSG)** to visualize data within AMP



You can also **use AWS Distro for OpenTelemetry (ADOT)** to ingest application metrics from your environment with AMP

Grafana



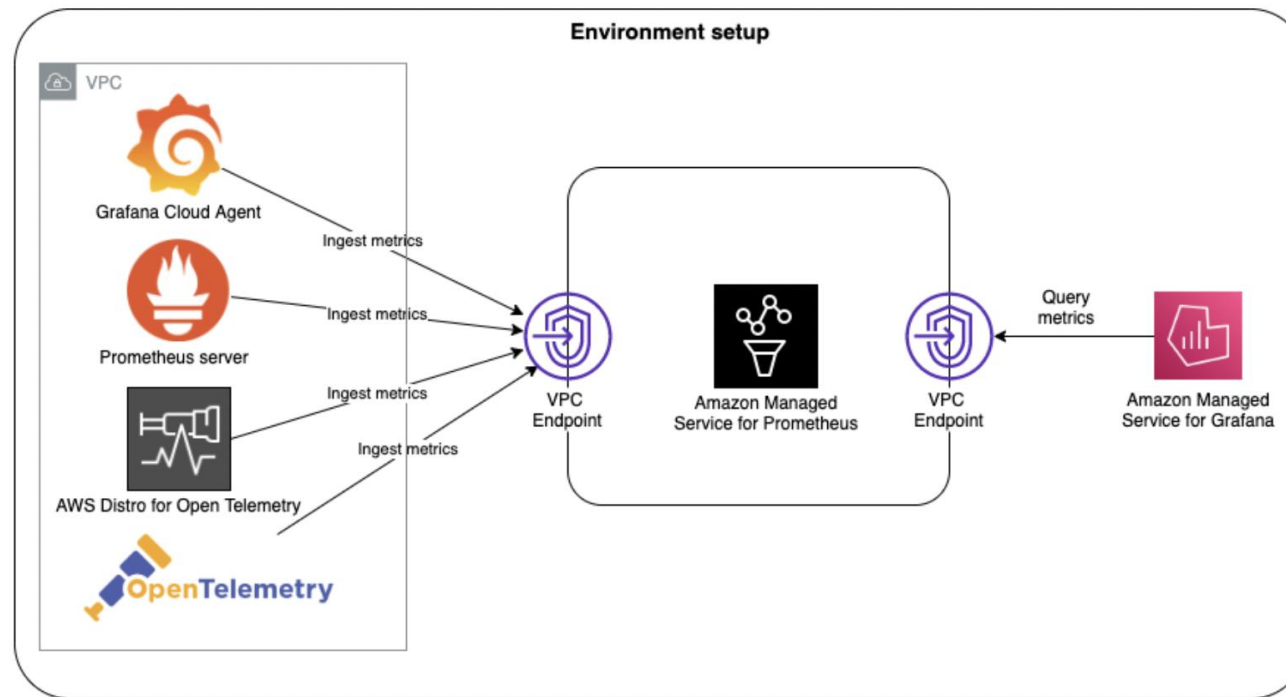
Grafana is an open source **analytics and interactive visualization** web application
Grafana is commonly used along with a time-series database like: InfluxDB, Prometheus, or Graphite



Amazon Managed Service for Grafana



Amazon Managed Grafana (AMG) is a fully managed and secure data visualization service that you can use to instantly query, correlate, and visualize operational metrics, logs, and traces from multiple sources.



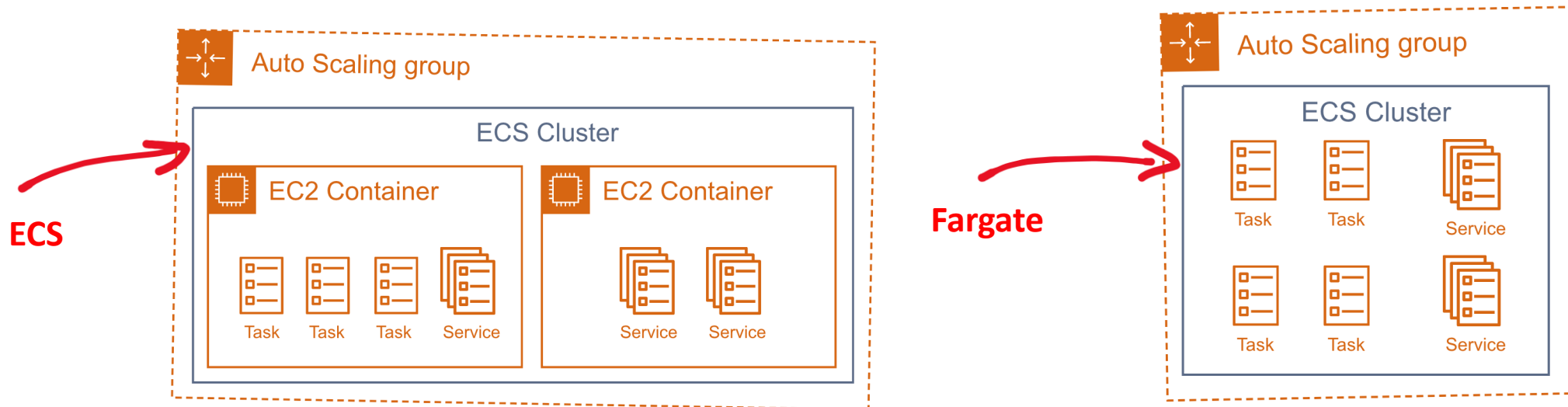
Introduction to Fargate



ECS Fargate is **serverless orchestration container service**

AWS handles the underlying server, so you don't need to manage EC2 instances

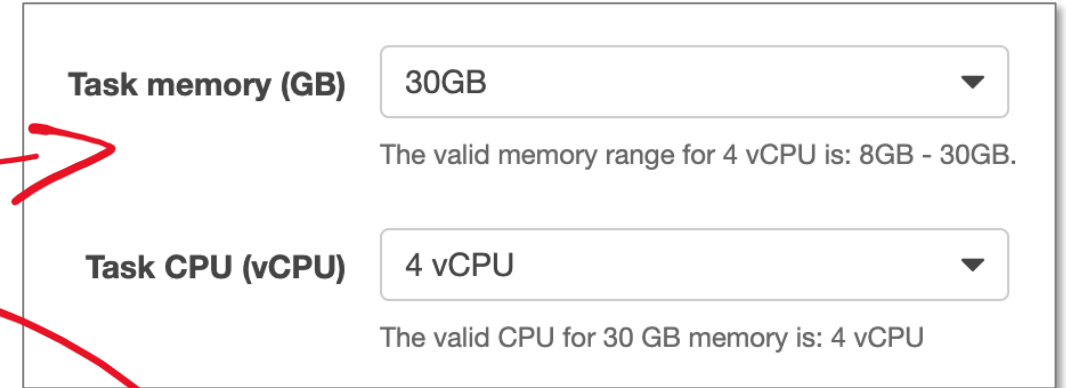
- No EC2 provisioning is required; you can create an **empty ECS cluster** and launch tasks directly
- Automatically handles container scaling
- Charged by **duration and consumption** (minimum **1-minute** billing, then per second)
- Requires **AWSVPC network mode**, which provides an Elastic Network (ENI) for each task group in the VPC
- Must use **IP addresses** instead of hostnames for Fargate tasks



Configuring Fargate Tasks

- In your Fargate Task Definition, you **define** the **memory** and **vCPU**.
- You will then add your containers and **allocate** the memory and vCPU required for each
- When you run the Task, you can choose what VPC and subnet it will run in
- **Apply a Security Group to a Task**
- Apply an IAM role to the Task

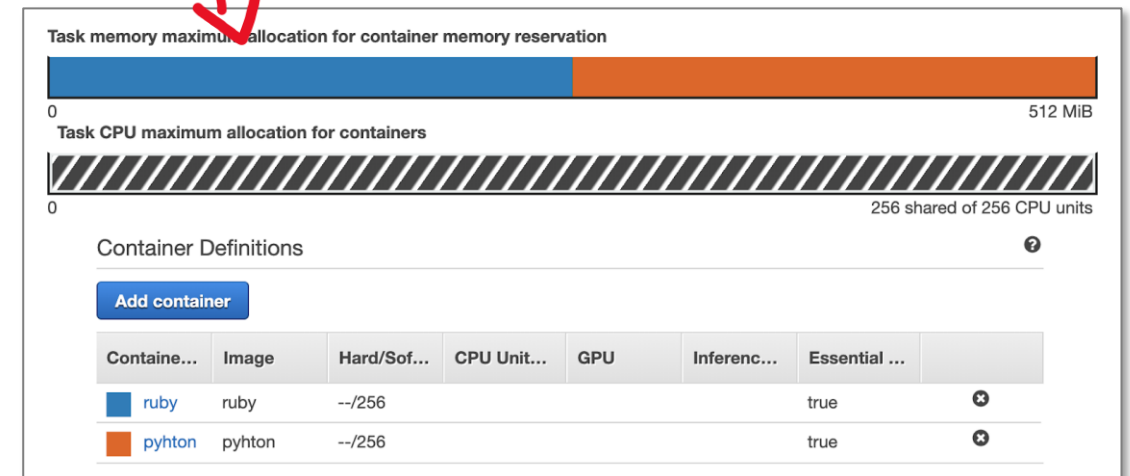
**You can apply SG and IAM roles for both ECS and Fargate Tasks and Services*



Task memory (GB) 30GB
The valid memory range for 4 vCPU is: 8GB - 30GB.

Task CPU (vCPU) 4 vCPU
The valid CPU for 30 GB memory is: 4 vCPU

Red arrows from the text 'memory' and 'vCPU' in the first bullet point point to these fields. Another red arrow from the text 'allocate' in the second bullet point points to the container definitions table below.



Task memory maximum allocation for container memory reservation

Task CPU maximum allocation for containers

Container Definitions

Add container

Containe...	Image	Hard/Sof...	CPU Unit...	GPU	Inferenc...	Essential ...	
ruby	ruby	--/256				true	✖
python	pyhton	--/256				true	✖

512 MiB

256 shared of 256 CPU units

Fargate Compute Configuration

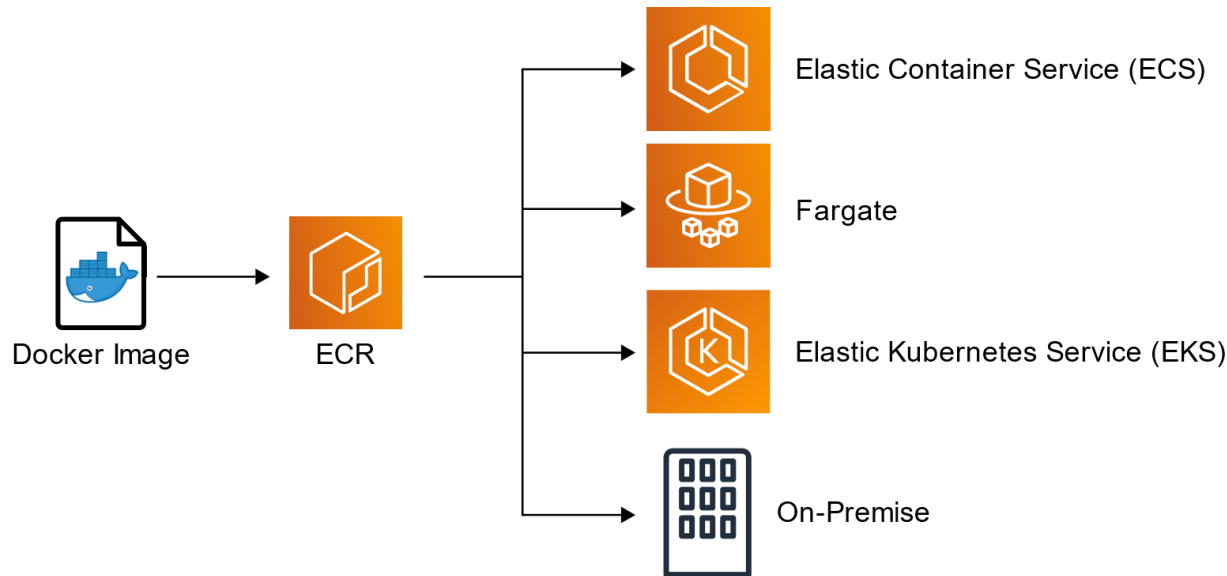
When you choose vCPU and Memory for Fargate, you must choose from these configurations

vCPU			
0.25 vCPU	0.5 GB, 1 GB, and 2 GB		
	Min	Max	Increments
0.5 vCPU	1 GB	4 GB	in 1 GB increments
1 vCPU	2 GB	8 GB	
2 vCPU	4 GB	16 GB	
4 vCPU	8 GB	30 GB	
8 vCPU	16 GB	60 GB	in 4 GB increments
16 vCPU	32 GB	120 GB	in 8 GB increments

Elastic Container Registry (ECR)

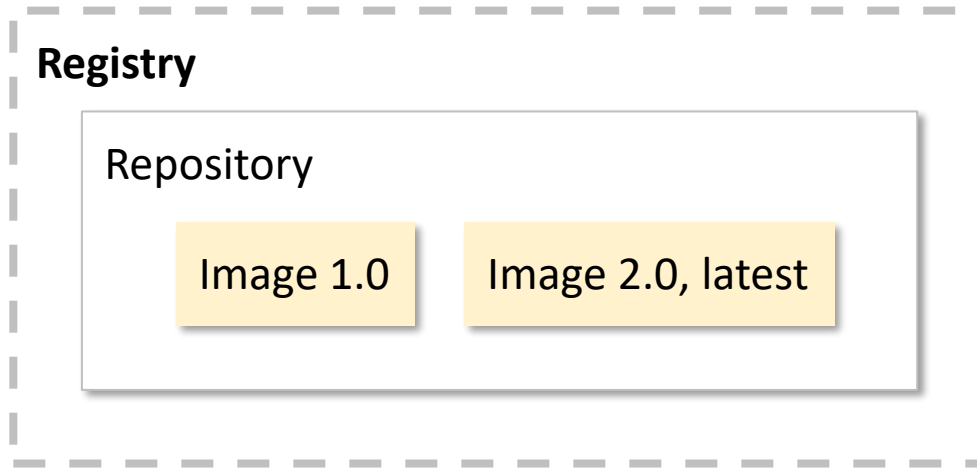


A fully-managed Docker container registry that makes it easy for developers to **store**, **manage**, and **deploy** Docker container images.



- ECR lets you store **Docker** and **Open Container Initiative (OCI) images** and artifacts.
- You can control private register access via Register policy
- You can control private repo access via a Repo Policy
- You can scan images on push to identify software vulnerabilities
- Private image replication allows you to have cross-account and cross-region images
- With ECR you can create a Pull Through Cache to sync the contents of an upstream registry
- ECR encrypts-at-rest repo images
- Amazon ECR lifecycle allow you to manage automate the cleaning up container images
- You can sign images via AWS Signer to ensure images are from your trusted developers
- Tags can be set to Mutable or Immutable

Elastic Container Registry (ECR)



- A **registry** contains multiple repo(sitories)
- A **repo** contains multiple images
- A **image** can have multiple tags
- A **tag** points to a specific image version
 - Eg. 1.0, latest

ECR supports:

- **Public registries** — accessible to anyone
- **Private registries** — only accessible to those within the AWS Account
 - Control access via **Register Policy**
 - ecr:ReplicateImage
 - ecr:BatchImportUpstreamImage
 - ecr>CreateRepository
 - Control access via **Repo Policy**
 - ecr:DescribeImages
 - ecr:DescribeRepositories

Elastic Container Registry (ECR)

To push to ECR you must first **remotely login using docker**.
You need to have authorization token and have AWS Credentials
configure in your environment to get said token

Build your image

Tag your image

```
# Log into ECR
aws ecr get-login-password --region ca-central-1 | docker login \
--username AWS \
--password-stdin 012345678912.dkr.ecr.ca-central-1.amazonaws.com

# Build Image
docker build -t my-image .

# Tag Image
docker tag my-image:latest <ECR_URI>:latest

# Push to ECR
docker push <ECR_URI>:latest
```

Push your image

Elastic Container Registry (ECR)

Image tag mutability feature prevent image tags from being overwritten

```
aws ecr create-repository \  
--repository-name name \  
--image-tag-mutability IMMUTABLE \  
--region ca-central-1
```

When tag immutability is turned on for a repository, this affects all tags and you cannot make some tags immutable while others aren't.

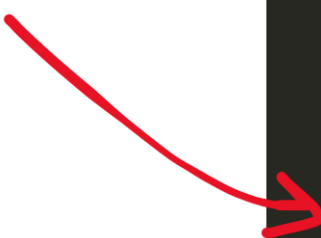


Immutable tags is a best practice because if there was a security vulnerability with a specific image you can rollback to previous image or preserve the history of vulnerabilities

ImageTagAlreadyExistsException error is returned if you attempt to push an image with a tag that is already in the repository

Elastic Container Registry (ECR)

ECR Lifecycle policy can be used to expire old images based on specific criteria.



```
{
  "rules": [
    {
      "rulePriority": 1,
      "description": "Expire images older than 14 days",
      "selection": {
        "tagStatus": "tagged",
        "tagPatternList": ["prod*"],
        "countType": "sinceImagePushed",
        "countUnit": "days",
        "countNumber": 14
      },
      "action": {
        "type": "expire"
      }
    }
  ]
}
```

AWS Copilot CLI



AWS Copilot CLI lets you build, release and operate production ready containerized applications on AWS App Runner, Amazon ECS, and AWS Fargate via easy-to-use CLI commands.

<https://aws.github.io/copilot-cli/>

```
~/my-app >>> copilot init
Note: It's best to run this command in the root of your Git repository.
Welcome to the Copilot CLI! We're going to walk you through some questions
to help you get set up with an application on ECS. An application is a collec
tion of
containerized services that operate together.

Your workspace is registered to application my-app.
Service type: Load Balanced Web Service
Service name: frontend

Which Dockerfile would you like to use for frontend? [Use arrows to move,
type to filter, ? for more help]
> api/Dockerfile
  tracker/Dockerfile
```

AWS Copilot CLI

Copilot can be easily install via **hombrew**

```
brew install aws/tap/copilot-cli
```

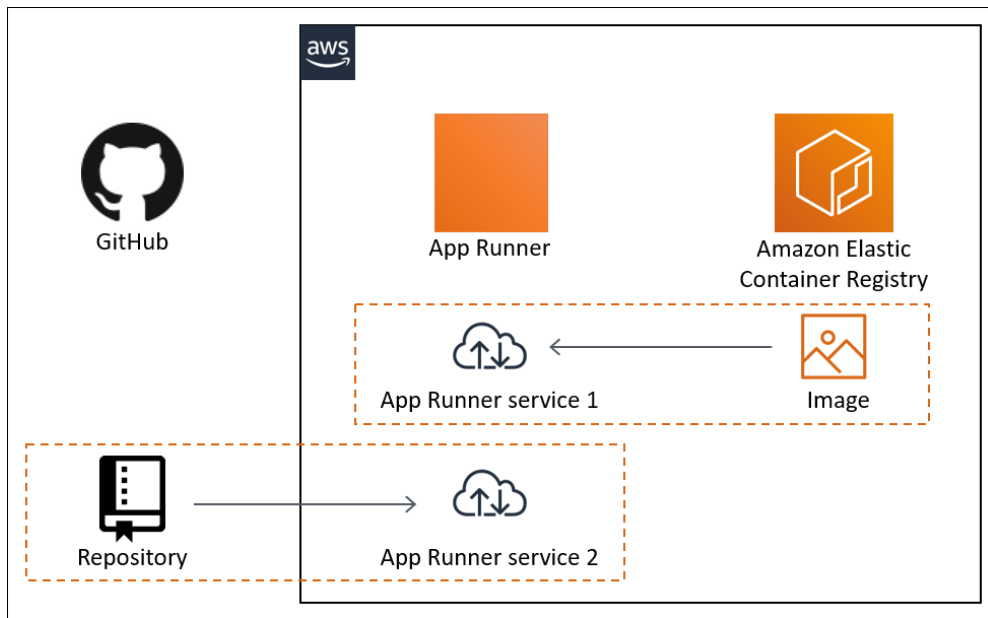
AWS Copilot structure your container workloads with the following components:

- **Applications** — a collection of services and environment, represent a workload
- **Environments** — represents a version of a service for a specific use case eg. Development and Production
- **Services** — The type of service you want to deploy, long running tasks
- **Jobs** — ephemeral Amazon ECS tasks that are triggered by an event. (short running tasks)
- **Pipelines** — a release pipeline for CI/CD connection to a Version Control Systems (VCS) enabling GitOps workflow

AWS App Runner



AWS App Runner is an AWS service that provides a fast, simple, and cost-effective way to deploy from source code or a container image directly to a scalable and secure web application in the AWS Cloud.



- App Runner is Platform as a Service (PaaS) specifically for containers
- It is easier to use than both Elastic Beanstalk (EB) and Fargate
- It directly integrates with Version Control Systems eg. VCS
- It has simplified networking
 - Choose between a Private or Public Endpoint
- It has automatic scaling
- Xray Tracing can be easily turned on.
- Its use cases are for simple apps or APIs for rapid deployment of containers
- Easily connects to a container registry eg. ECR
- Has TCP or HTTP health checks
- Supports easy integration with Secrets Manager
- Supports Custom Domains
- KMS can be used for at-rest-encryption
- You pay based on:
 - vCPUs per hour
 - GB of storage per hour
 - Automatic deploys (\$1 app per month)
 - Build time per minute

AWS App Runner

When you choose CPU and Memory for AWS App Runner you must choose from this combination

CPU	Memory
0.25 vCPU	0.5 GB, 1 GB
0.5 vCPU	1 GB
1 vCPU	2 GB, 3 GB, 4 GB
2 vCPU	4 GB, 6 GB
4 vCPU	8 GB, 10 GB, 12 GB

AWS App Runner

Example of **deploying a container** to AWS AppRunner



```
aws apprunner create-service \
--cli-input-json '{
  "ServiceName": "MySimpleWebApp",
  "SourceConfiguration": {
    "AuthenticationConfiguration": {},
    "ImageRepository": {
      "ImageIdentifier": "111122223333.dkr.ecr.us-west-2.amazonaws.com/my-web-app:latest",
      "ImageRepositoryType": "ECR",
      "ImageConfiguration": {"Port": "80"}
    }
  },
  "InstanceConfiguration": {
    "Cpu": "1024",
    "Memory": "2048",
    "InstanceRoleArn": "arn:aws:iam::111122223333:role/AppRunnerECRAccessRole"
  }
}'
```

RedHat OpenShift

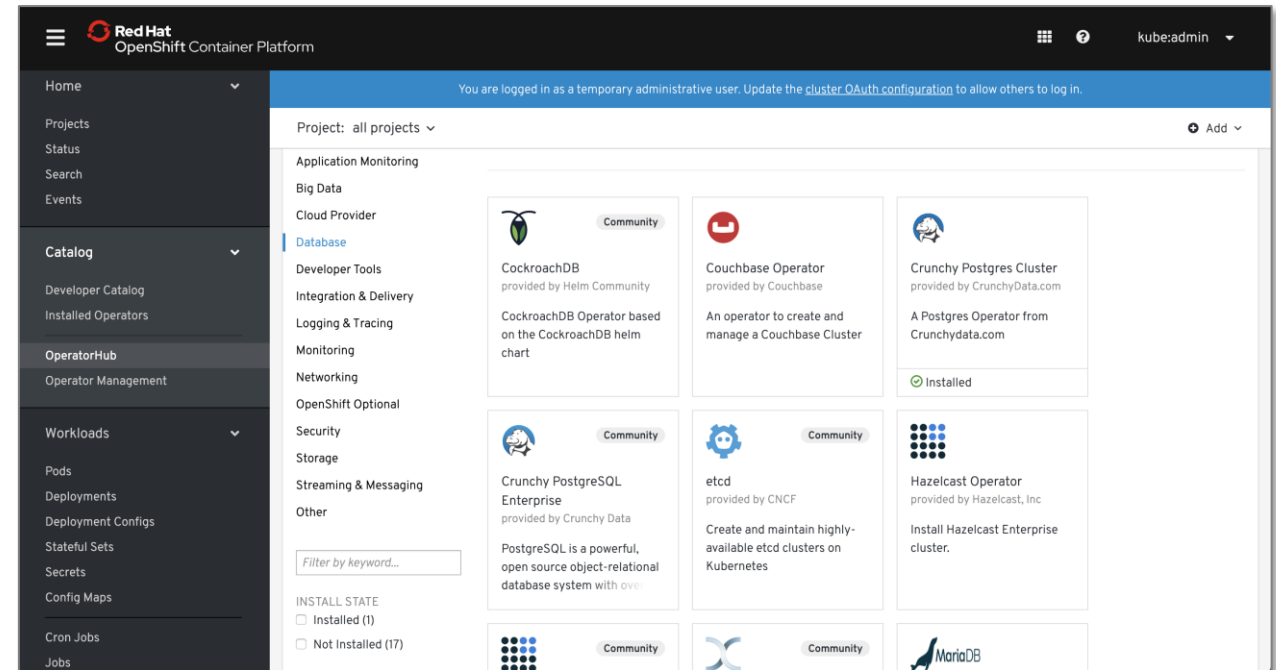


Red Hat OpenShift *Platform as a Service for K8s*

- OpenShift is K8s with a commercial platform by RedHat built on top.
- Kubectl is extended with additional functionality with the oc CLI
- quickly deploy local code to a remote OpenShift cluster via `odo`
- A quality assurance pipeline built into the platform
- Fixing critical bugs earlier instead of waiting for the next K8s release
- Using Redhat CoreOS (an operating system optimized for running containers)
- OperatorHub, an automated installation tool (one-click marketplace)
- CodeReady workspaces, Cloud Developer Environment for Kubernetes
- Graphical UI developer console

Red Hat OpenShift on AWS (ROSA)

Provides an integrated experience with OpenShift.
You utilize AWS compute, but leverage the RedHat PaaS for K8s



App2Container

AWS App2Container (A2C) is a command line tool to help **you lift and shift applications** that run in your on-premises data centers or on virtual machines, so that they run in containers that are managed by ECS, EKS, or App Runner. A2C is free to use.

App2Container supports the following application types:

- Java applications (Linux)
- ASP.NET applications (Windows, Linux)

Setup

- Create ECR repos to store the eventual images
- Launch an EC2 instance that will handle the migration
- Install your app on to the EC2 instance
- Install Docker on to the EC2 Instance
- Install App2Container onto the EC2 instance

Containerization Steps:

- Analyze the app
- Generate Containerization Recommendations
- Package App into Containers
- Push the Containers to ECR

```
# Analyze App
sudo app2container init
./app2container analyze \
--source <path-to-application> \
--output report.json

# Generate Containerization Recommendations
./app2container recommend \
--source <path-to-application> \
--app-name <application-name> \
--output recommendations.json

# Package App into Containers
./app2container package \
--source <path-to-application> \
--recommendations recommendations.json \
--output-dir <output-directory>

# Push the containers
docker push <repository-url>/<image-name>:<tag>
```